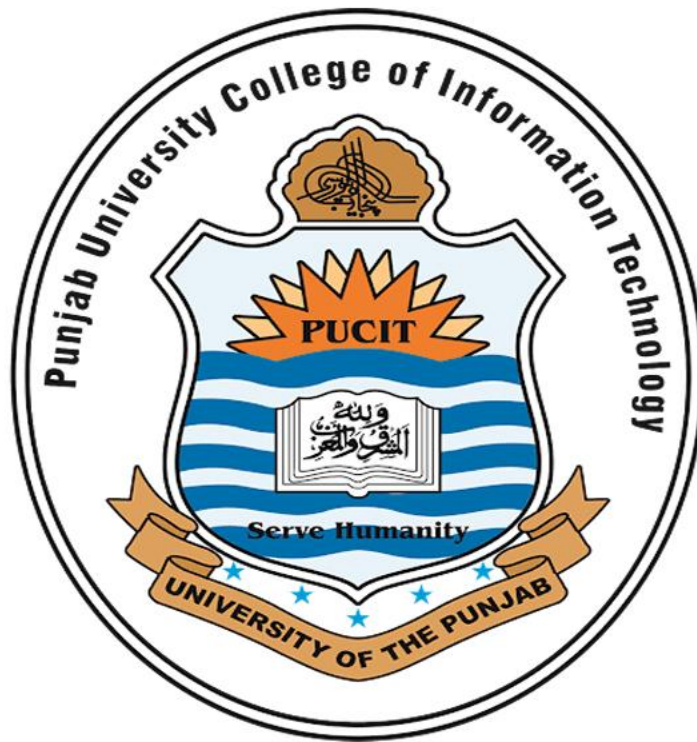




Course: Database Systems

Submitted to: Dr. Asif Sohail

Submitted by:

- Muhammad Abu Hurairah Nazar (BITF24A037)
- Muhammad Zain Khan (BITF24A031)
- Abdul Ghani (BITF24A003)

Work: Project Report

Dated: 22 December, 2025

PUNJAB UNIVERSITY COLLEGE OF INFORMATION TECHNOLOGY

Database Systems – Project Report

1. Introduction

Modernizing library data systems is a major challenge today. Libraries are shifting from physical shelves to hybrid setups handling both real items and digital info, so the database needs to handle strong data accuracy, complex rules, and fast searches.

This report covers the full design, rationale, and setup of the Library Management Database. It powers a fast library app, fixing the old issues with file-based storage.

Using a relational model and Oracle SQL's advanced features (like procedures and triggers), LMS-DB centralizes data on books, users, loans, and finances. It enforces rules (no invalid checkouts, no duplicates), links everything properly, and streamlines tasks like adding books, cataloging, circulation, and financial reporting.

Overall, it makes libraries run smoother, reduces errors, and scales easily as collections and users grow.

1.1 System Architecture and Operational Objectives

The main goal of the LMS-DB is to replace scattered spreadsheets and text files with a solid relational database that actually works reliably.

Old systems struggle with growth and can't properly handle rules—like stopping the same book copy from being checked out to two people at once. This new setup tracks everything in a library's lifecycle: from adding a new title and cataloging it, through lending it to patrons, right up to removing or replacing it.

The system covers these key areas:

1. **Catalog Management:** It separates the book's content (Title) from its physical copies (Items), so you can track multiple copies of the same book without repeating data.
2. **User Management:** It handles detailed patron profiles and different types (Undergrads, Faculty, Staff), applying the right borrowing limits and loan durations for each.
3. **Circulation:** The core engine processes checkouts, returns, and renewals, strictly enforcing rules on availability and who can borrow what.
4. **Fines & Payments:** It automatically calculates overdue fines using smart calendar logic that skips weekends, holidays, and non-working days.

5. **Reservations:** A queue system manages holds for items that are currently out, making sure high-demand books go to people in fair order.

1.2 User Levels and Access Control Mechanisms

The database security works on a simple but strong role-based system. The app handles login, but the database itself uses special views and stored procedures to control what each type of user can actually do.

- **System Admins:** They have full access to everything. Admins set up key settings like daily fine rates, borrowing limits for different user types, and the holiday calendar.
- **Librarians & Staff:** They handle day-to-day work through safe, predefined procedures. They can add new members, catalog books, process checkouts and returns, but they can't mess with the database structure or see audit logs.
- **Patrons (Library Users):** Regular members get read-only access to user-friendly views. They can search the catalog, check if a book is available, see their own borrowing history, and view any fines they owe.

2. Requirement Analysis and Conceptual Design

The initial phase of the database design process involves a rigorous analysis of the functional requirements and the translation of these needs into a conceptual model. This section details the "Top-Down" approach, starting with the identification of high-level entities and refining them into a structured Entity-Relationship (ER) model.

HOLIDAYS		
DATE	HOLIDAY_DATE	PK
VARCHAR	DESCRIPTION	

PUBLISHER		
NUMBER	PUB_ID	PK
VARCHAR	PUB_NAME	
VARCHAR	ADDRESS	

AUTHOR		
NUMBER	AUTHOR_ID	PK
VARCHAR	FULL_NAME	
VARCHAR	BIO	

SHELF		
NUMBER	SHELF_ID	PK
NUMBER	FLOOR_NO	
VARCHAR	aisle_code	
VARCHAR	CATEGORY	

BOOK		
VARCHAR	ISBN	PK
VARCHAR	TITLE	
NUMBER	PUB_ID	FK
NUMBER	PUB_YEAR	
NUMBER	EDITION	

MEMBERSHIP_TYPE		
NUMBER	TYPE_ID	PK
VARCHAR	TYPE_NAME	
NUMBER	MAX_BOOKS	
NUMBER	LOAN_DAYS	
NUMBER	DAILY_FINE	

BOOK_AUTHOR		
VARCHAR	ISBN	PK,FK
NUMBER	AUTHOR_ID	PK,FK

BOOK_COPY		
NUMBER	COPY_ID	PK
VARCHAR	ISBN	FK
NUMBER	SHELF_ID	FK
VARCHAR	STATUS	

MEMBER		
NUMBER	MEMBER_ID	PK
VARCHAR	NAME	
VARCHAR	EMAIL	
VARCHAR	PHONE	
NUMBER	TYPE_ID	FK
VARCHAR	STATUS	

LOAN_TRANSACTION		
NUMBER	TRANS_ID	PK
NUMBER	COPY_ID	FK
NUMBER	MEMBER_ID	FK
DATE	ISSUE_DATE	
DATE	DUE_DATE	
DATE	RETURN_DATE	
VARCHAR	STATUS	

RESERVATION		
NUMBER	RES_ID	PK
VARCHAR	ISBN	FK
NUMBER	MEMBER_ID	FK
DATE	RES_DATE	
VARCHAR	STATUS	

FINE		
NUMBER	FINE_ID	PK
NUMBER	TRANS_ID	FK
NUMBER	AMOUNT	
VARCHAR	PAID_STATUS	

publishes

wrote

has author

stores

has inventory

is reserved

borrow

is borrowed in

incurs

categorizes

requests

2.1 Functional Requirements Analysis

The system needs to handle a few key groups of data smartly:

Books & Copies: Many basic library systems mix up a book's title/details with its physical copies, leading to tons of repeated info. For example, if a library has 10 copies of *Introduction to Algorithms*, you don't want the same author, publisher, and ISBN repeated 10 times. Instead, we separate it: one "Title" record for the book info (ISBN, author, etc.), and separate "Copy" records for each physical book (barcode, shelf location, condition). This cuts redundancy and makes updates way easier.

Patron Rules: Different users get different borrowing privileges. Faculty might borrow 10 items for 90 days, while students get 5 items for 14 days. We store these rules in a "Membership Type" table that links to each member's record, so the system can automatically apply the right limits.

Circulation & Transactions: This is the busy part. Every checkout links a specific member to a specific book copy. The system prevents double-borrowing by marking copies as "Issued" until they're returned. It also keeps a full history of every loan for reports and stats.

Fines: Fines are calculated based on how many days an item is overdue (due date vs. return date). But we don't charge for weekends or holidays, so the system includes a "Holidays" table and smart functions to count only actual business days.

2.2 Entity-Relationship Model (Conceptual Design)

The ER model is basically the blueprint that shows how all the pieces of the library database fit together. Here's a straightforward breakdown of the main entities and what they represent:

- **Publisher:** The companies that publish books. It includes their ID, name, and address.
- **Author:** The people who write the books. It has an ID, name, and a short bio.
- **Book (Title):** The core record for a book's content (the abstract version). It covers ISBN, title, year published, and category.
- **Book Copy:** Each physical book in the library. It has a unique barcode (Copy ID), status (like Available, Issued, or Lost), and shelf location.
- **Member:** The library users (patrons). It includes their ID, name, email, phone, and registration date.
- **Membership Type:** Defines the borrowing rules for different kinds of users. It has a type ID, description, max books they can borrow, loan period in days, and daily fine rate.

- **Loan (Issue):** Tracks every checkout. It records the transaction ID, issue date, due date, and return date.
- **Fine:** Handles overdue penalties tied to a specific loan. It includes a fine ID, amount owed, and whether it's paid or not.
- **Reservation:** Manages holds for books that are currently checked out. It has a reservation ID, request date, and status (like active or fulfilled).

Relationships and Cardinality:

- **Author-Book (M:M):** A book can have multiple authors, and an author can write multiple books. This requires an associative entity, **BOOK_AUTHOR**.
- **Publisher-Book (1:M):** A publisher produces many books, but a specific ISBN edition is linked to one publisher.
- **Book-Book_Copy (1:M):** One bibliographic title can be associated with multiple physical copies on the shelves.
- **Member-Membership_Type (M:1):** Many members belong to a single category (e.g., 'Student'), which defines their privileges.
- **Member-Loan (1:M):** A member can perform multiple borrowing transactions over time.
- **Book_Copy-Loan (1:M):** A specific copy can be borrowed many times over its lifespan (though only once at a time, enforced by status).
- **Loan-Fine (1:1):** A loan transaction *may* generate a fine record if overdue. While a member can have many fines, a specific fine is linked to a specific loan instance.

3. Construction of the Relational Schema

To build the relational schema for the LMS-DB, we're using a "bottom-up" approach—just like in the reference project. This means starting with a full list of all the individual data points (attributes) we need to track, figuring out how they depend on each other, and then organizing them into clean, normalized tables.

3.1 Identifying All the Attributes

Here's the complete list of data points we've identified so far for the library system:

- **Book-related:** ISBN, Title, Edition, Publication Year, Category Name, Publisher Name, Publisher Address, Author Name
- **Physical copy details:** Copy Barcode, Purchase Date, Shelf Code, Floor Number
- **Member info:** Member ID, Member Name, Member Email, Member Phone, Member Address, Membership Type, Max Book Limit
- **Loan transactions:** Issue Date, Due Date, Return Date
- **Fines:** Fine Amount, Fine Payment Date
- **Reservations:** Reservation Date
- **Staff info:** Staff ID, Staff Name
- **Holidays:** Holiday Date

3.2 Analysis of Functional Dependencies

Functional dependencies (FDs) are basically the rules that show how one piece of data determines others. Spotting these helps us group attributes into tables without redundancy or weird update issues during normalization.

Here are the key dependencies we've identified:

- **ISBN → Title, Edition, Publication Year, Publisher ID**
- **Publisher ID → Publisher Name, Publisher Address**
- **Author ID → Author Name**
- **Copy ID (Barcode) → ISBN, Shelf ID, Purchase Date, Current Status**
- **Member ID → Name, Email, Address, Membership Type ID**
- **Membership Type ID → Description, Max Books Limit, Loan Period, Daily Fine Rate**
- **Transaction ID → Copy ID, Member ID, Issue Date, Due Date**
- **Shelf ID → Floor Number, Aisle Code, Shelf Category**

3.3 Grouping of Attributes into Relations

Before normalization, we organize these attributes into their primary groups based on the entities they describe.

1. **PUBLISHER** (Pub_ID, Pub_Name, Address, Contact_No)
 2. **AUTHOR** (Author_ID, Full_Name, Bio)
 3. **BOOK** (ISBN, Title, Pub_ID, Pub_Year, Edition, Category_ID)
 4. **BOOK_AUTHOR** (ISBN, Author_ID)
 5. **SHELF** (Shelf_ID, Floor_No, Aisle_Code, Category)
 6. **BOOK_COPY** (Copy_ID, ISBN, Shelf_ID, Status)
 7. **MEMBERSHIP_TYPE** (Type_ID, Type_Name, Max_Books, Loan_Days, Daily_Fine)
 8. **MEMBER** (Member_ID, Name, Email, Phone, Type_ID, Status, Reg_Date)
 9. **LOAN_TRANSACTION** (Trans_ID, Copy_ID, Member_ID, Issue_Date, Due_Date, Return_Date, Status)
 10. **FINE** (Fine_ID, Trans_ID, Amount, Paid_Status, Pay_Date)
 11. **RESERVATION** (Res_ID, ISBN, Member_ID, Res_Date, Status)
 12. **HOLIDAYS** (Holiday_Date, Description)
-

4. Normalization Process

This section demonstrates the normalization of each relation defined above, ensuring they satisfy 1NF, 2NF, 3NF, and BCNF to eliminate redundancy and anomalies.

PUBLISHER (Pub_ID, Pub_Name, Address, Contact_No)

- **1NF:** All attributes (Name, Address, Contact) contain atomic values. There are no repeating groups. Thus, it is in 1NF.
- **2NF:** The Primary Key is Pub_ID (a single attribute). Partial dependency is impossible with a single-attribute PK. Thus, it is in 2NF.
- **3NF:** There are no transitive dependencies. Address and Contact_No depend directly on the Publisher, not on each other. Thus, it is in 3NF.
- **BCNF:** The only determinant is Pub_ID, which is a candidate key. No overlapping keys exist. Thus, it is in BCNF.

AUTHOR (Author_ID, Full_Name, Bio)

- **1NF:** Full_Name and Bio are atomic. It is in 1NF.
- **2NF:** The Primary Key is Author_ID (single attribute). No partial dependencies exist. It is in 2NF.
- **3NF:** Bio describes the author identified by Author_ID. No non-key attribute determines another. It is in 3NF.
- **BCNF:** Author_ID is the superkey for all functional dependencies. It is in BCNF.

BOOK (ISBN, Title, Pub_ID, Pub_Year, Edition)

- **1NF:** All attributes are atomic. (Note: Multiple authors were removed to the BOOK_AUTHOR table to satisfy 1NF here). It is in 1NF.
- **2NF:** The Primary Key is ISBN (single attribute). All non-key attributes (Title, Pub_ID, etc.) depend fully on ISBN. It is in 2NF.
- **3NF:** Pub_ID is a Foreign Key. The details of the publisher (Name, Address) are stored in the PUBLISHER table, not here. Therefore, Pub_Name does not transitively depend on ISBN via Pub_ID. It is in 3NF.
- **BCNF:** ISBN is the unique identifier (Candidate Key) for a book edition. All dependencies flow from ISBN. It is in BCNF.

BOOK_AUTHOR (ISBN, Author_ID)

- **1NF:** Contains only atomic Foreign Keys linking Books and Authors. It is in 1NF.
- **2NF:** The Primary Key is composite (ISBN, Author_ID). There are no non-key attributes in this relation (it is purely associative), so partial dependency is not possible. It is in 2NF.
- **3NF:** With no non-key attributes, there can be no transitive dependencies. It is in 3NF.
- **BCNF:** The key is the entire tuple. Trivial dependencies apply. It is in BCNF.

SHELF (Shelf_ID, Floor_No, Aisle_Code, Category)

- **1NF:** Attributes are atomic. It is in 1NF.
- **2NF:** PK is Shelf_ID. No partial dependencies. It is in 2NF.
- **3NF:** Floor_No and Aisle_Code describe the physical location of the shelf. One does not determine the other (an aisle spans floors, a floor has many aisles). Dependencies are on Shelf_ID. It is in 3NF.
- **BCNF:** Shelf_ID is the superkey. It is in BCNF.

BOOK_COPY (Copy_ID, ISBN, Shelf_ID, Status)

- **1NF:** Atomic attributes. It is in 1NF.
- **2NF:** PK is Copy_ID. No partial dependencies. It is in 2NF.
- **3NF:** ISBN and Shelf_ID are Foreign Keys. The Title details and Shelf details are not stored here, preventing transitive dependencies. Status depends on the specific copy. It is in 3NF.
- **BCNF:** Copy_ID uniquely identifies a physical item. ISBN is not a candidate key here (many copies share an ISBN). The Copy_ID is a superkey. It is in BCNF.

MEMBERSHIP_TYPE (Type_ID, Type_Name, Max_Books, Loan_Days, Daily_Fine)

- **1NF:** Atomic values. It is in 1NF.
- **2NF:** PK is Type_ID. No partial dependencies. It is in 2NF.
- **3NF:** Rules like Loan_Days depend on the Type_ID (e.g., Student vs. Staff). No transitive dependencies exist between the non-key attributes. It is in 3NF.
- **BCNF:** Type_ID is the candidate key. It is in BCNF.

MEMBER (Member_ID, Name, Email, Phone, Type_ID, Status, Reg_Date)

- **1NF:** Atomic attributes. It is in 1NF.
- **2NF:** PK is Member_ID. No partial dependencies. It is in 2NF.
- **3NF:** Type_ID is a foreign key. The borrowing limits (Max_Books) are stored in MEMBERSHIP_TYPE, not here. If we had stored Max_Books here, it would be a transitive dependency (Member_ID -> Type_ID -> Max_Books). Since we separated it, it is in 3NF.
- **BCNF:** Member_ID is the superkey. Email is a candidate key (Unique), and Email -> Member_ID is valid. It is in BCNF.

LOAN_TRANSACTION (Trans_ID, Copy_ID, Member_ID, Issue_Date, Due_Date, Return_Date, Status)

- **1NF:** Atomic attributes. It is in 1NF.
- **2NF:** PK is Trans_ID. No partial dependencies. It is in 2NF.
- **3NF:** Due_Date is calculated but stored for persistence. It depends on Issue_Date and the Member's rules, but within the context of a transaction record, it is considered directly dependent on the Transaction event. It is in 3NF.
- **BCNF:** Trans_ID is the superkey. Copy_ID + Issue_Date could be a candidate key (a copy can only be issued once at a time), but Trans_ID covers this. It is in BCNF.

FINE (Fine_ID, Trans_ID, Amount, Paid_Status, Pay_Date)

- **1NF:** Atomic values. It is in 1NF.
- **2NF:** PK is Fine_ID. No partial dependencies. It is in 2NF.
- **3NF:** Amount depends on the Trans_ID (specifically the overdue duration), but is stored here as a finalized fiscal record. No transitive dependencies between non-key attributes. It is in 3NF.
- **BCNF:** Fine_ID is the superkey. It is in BCNF.

RESERVATION (Res_ID, ISBN, Member_ID, Res_Date, Status)

- **1NF:** Atomic values. It is in 1NF.
 - **2NF:** PK is Res_ID. No partial dependencies. It is in 2NF.
 - **3NF:** Status depends on the reservation lifecycle. No transitive dependencies. It is in 3NF.
 - **BCNF:** Res_ID is the superkey. It is in BCNF.
-

5. Data Dictionary and Schema Definition

This section provides a detailed technical specification of the database entities, defining data types, sizes, and integrity constraints. This corresponds to the physical design phase.

5.1 Configuration and Reference Entities

These tables store relatively static data that defines the rules and structure of the library system.

Table 1: MEMBERSHIP_TYPE Defines the various categories of library patrons and their associated privileges.

- **Primary Key:** TYPE_ID
- **Alternate Key:** TYPE_NAME

Attribute	Data Type	Constraints	Description
TYPE_ID	NUMBER(2)	PK	Unique identifier for the member category.
TYPE_NAME	VARCHAR2(20)	NOT NULL, UNIQUE	Descriptive name (e.g., 'Student', 'Faculty').
MAX_BOOKS	NUMBER(2)	CHECK (>0)	Maximum number of books a member can hold.
LOAN_DAYS	NUMBER(3)	NOT NULL	Standard loan duration in days.
DAILY_FINE	NUMBER(5,2)	DEFAULT 0.00	Fine amount per day for overdue items.

Table 2: SHELF Normalizes physical location data to facilitate inventory management.

- **Primary Key:** SHELF_ID

Attribute	Data Type	Constraints	Description
SHELF_ID	NUMBER(4)	PK	Unique identifier for a shelf unit.
FLOOR_NO	NUMBER(2)	NOT NULL	Floor number in the library building.
AISE_CODE	VARCHAR2(5)	NOT NULL	Alphanumeric code for the aisle.
CATEGORY	VARCHAR2(50)	NOT NULL	Genre or subject stored (e.g., 'Science').

Table 3: HOLIDAYS Stores dates that are excluded from fine calculations (e.g., public holidays).

- **Primary Key:** HOLIDAY_DATE

Attribute	Data Type	Constraints	Description
HOLIDAY_DATE	DATE	PK	The specific calendar date of the holiday.
DESCRIPTION	VARCHAR2(50)	NULL	Name of the holiday (e.g., 'New Year').

5.2 Bibliographic Entities

These tables manage the intellectual content of the library.

Table 4: PUBLISHER

- **Primary Key: PUB_ID**
- **Alternate Key: PUB_NAME**

Attribute	Data Type	Constraints	Description
PUB_ID	NUMBER(5)	PK	Unique publisher ID.
PUB_NAME	VARCHAR2(100)	NOT NULL, UNIQUE	Name of the publishing house.
ADDRESS	VARCHAR2(255)	NULL	Physical headquarters address.

Table 5: AUTHOR

- **Primary Key: AUTHOR_ID**

Attribute	Data Type	Constraints	Description
AUTHOR_ID	NUMBER(10)	PK	Unique author ID.
FULL_NAME	VARCHAR2(100)	NOT NULL	Full professional name of the author.
BIO	VARCHAR2(1000)	NULL	Brief biographical note.

Table 6: BOOK (Title) Represents the ISBN-level bibliographic record.

- **Primary Key:** ISBN
- **Foreign Key:** PUB_ID (Refs PUBLISHER)

Attribute	Data Type	Constraints	Description
ISBN	VARCHAR2(13)	PK	International Standard Book Number.
TITLE	VARCHAR2(255)	NOT NULL	Title of the work.
PUB_ID	NUMBER(5)	FK (Ref PUBLISHER)	Link to the publisher.
PUB_YEAR	NUMBER(4)	CHECK (1000-2100)	Year of publication.
EDITION	NUMBER(2)	DEFAULT 1	Edition number of the book.

Table 7: BOOK_AUTHOR Associative entity resolving the Many-to-Many relationship.

- **Primary Key:** Composite (ISBN, AUTHOR_ID)
- **Foreign Keys:** ISBN (Refs BOOK), AUTHOR_ID (Refs AUTHOR)

Attribute	Data Type	Constraints	Description
ISBN	VARCHAR2(13)	PK, FK	Link to Book.

Attribute	Data Type	Constraints	Description
AUTHOR_ID	NUMBER(10)	PK, FK	Link to Author.

5.3 Operational Entities

These tables manage the daily operations, inventory, and users.

Table 8: MEMBER

- **Primary Key:** MEMBER_ID
- **Foreign Key:** TYPE_ID (Refs MEMBERSHIP_TYPE)
- **Alternate Key:** EMAIL

Attribute	Data Type	Constraints	Description
MEMBER_ID	NUMBER(10)	PK	Unique patron ID.
NAME	VARCHAR2(100)	NOT NULL	Full name of the patron.
EMAIL	VARCHAR2(100)	UNIQUE, NOT NULL	Contact email (used for notifications).
PHONE	VARCHAR2(15)	NOT NULL	Contact phone number.
TYPE_ID	NUMBER(2)	FK (Ref MEMBER_TYPE)	Patron category.
STATUS	VARCHAR2(10)	CHECK (Active/Susp)	Account status.

Table 9: BOOK_COPY Represents the physical inventory item.

- **Primary Key:** COPY_ID
- **Foreign Keys:** ISBN (Refs BOOK), SHELF_ID (Refs SHELF)

Attribute	Data Type	Constraints	Description
COPY_ID	NUMBER(12)	PK	Unique barcode/accession number.
ISBN	VARCHAR2(13)	FK (Ref BOOK)	Link to bibliographic record.
SHELF_ID	NUMBER(4)	FK (Ref SHELF)	Physical location.
STATUS	VARCHAR2(20)	CHECK (Valid Status)	Current state (Available, Issued, Lost).

Table 10: LOAN_TRANSACTION

- **Primary Key:** TRANS_ID
- **Foreign Keys:** COPY_ID (Refs BOOK_COPY), MEMBER_ID (Refs MEMBER)

Attribute	Data Type	Constraints	Description
TRANS_ID	NUMBER(15)	PK	Unique transaction ID.
COPY_ID	NUMBER(12)	FK (Ref COPY)	The specific item borrowed.

Attribute	Data Type	Constraints	Description
MEMBER_ID	NUMBER(10)	FK (Ref MEMBER)	The borrower.
ISSUE_DATE	DATE	DEFAULT SYSDATE	Date of issue.
DUE_DATE	DATE	NOT NULL	Calculated return deadline.
RETURN_DATE	DATE	NULL	Actual return date.
STATUS	VARCHAR2(10)	CHECK (Open/Closed)	Transaction status.

Table 11: FINE

- **Primary Key:** FINE_ID
- **Foreign Key:** TRANS_ID (Refs LOAN_TRANSACTION)

Attribute	Data Type	Constraints	Description
FINE_ID	NUMBER(15)	PK	Unique fine ID.
TRANS_ID	NUMBER(15)	FK (Ref LOAN)	Associated loan.
AMOUNT	NUMBER(8,2)	NOT NULL	Calculated fine amount.
PAID_STATUS	VARCHAR2(10)	CHECK (Paid/Unpaid)	Payment status.

6. Physical Implementation (DDL)

This section details the Data Definition Language (DDL) commands required to implement the schema in Oracle SQL. The script includes the creation of sequences for surrogate keys and the table definitions with all integrity constraints.

6.1 Sequence Creation

Sequences are used to generate unique identifiers for primary keys, ensuring concurrency-safe insertions.

- CREATE SEQUENCE seq_pub_id START WITH 1 INCREMENT BY 1;
- CREATE SEQUENCE seq_author_id START WITH 1 INCREMENT BY 1;
- CREATE SEQUENCE seq_shelf_id START WITH 1 INCREMENT BY 1;
- CREATE SEQUENCE seq_member_id START WITH 1000 INCREMENT BY 1;
- CREATE SEQUENCE seq_copy_id START WITH 50000 INCREMENT BY 1;
- CREATE SEQUENCE seq_trans_id START WITH 100000 INCREMENT BY 1;
- CREATE SEQUENCE seq_fine_id START WITH 1 INCREMENT BY 1;

6.2 Table Definition Scripts

The creation order is critical to satisfy Foreign Key constraints; parent tables must be created before child tables.

Step 1: Reference Tables

```
CREATE TABLE MEMBERSHIP_TYPE (  
    TYPE_ID NUMBER(2) CONSTRAINT pk_mem_type PRIMARY KEY,  
    TYPE_NAME VARCHAR2(20) NOT NULL CONSTRAINT uq_type_name UNIQUE,  
    MAX_BOOKS NUMBER(2) NOT NULL CONSTRAINT chk_max_books CHECK (MAX_BOOKS > 0),  
    LOAN_DAYS NUMBER(3) NOT NULL CONSTRAINT chk_loan_days CHECK (LOAN_DAYS > 0),  
    DAILY_FINE NUMBER(5,2) DEFAULT 0.00 NOT NULL  
);
```

☒ Autocommit Rows: 10 Save Run

SELECT * FROM MEMBERSHIP_TYPE

[Results](#) [Explain](#) [Describe](#) [Saved SQL](#) [History](#)

TYPE_ID	TYPE_NAME	MAX_BOOKS	LOAN_DAYS	DAILY_FINE
2	Faculty	15	90	0
5	Researcher	20	120	0
6	Junior	2	7	.1
7	Senior Citizen	5	21	0
8	Corporate	10	14	2
9	Guest	1	7	5
1	Student	5	14	.5
3	Staff	10	30	.25
4	Alumni	3	14	1
10	Lifetime	5	30	0

10 rows returned in 0.01 seconds [Download](#)

Application Express 4.0.2.00.09

```

CREATE TABLE SHELF (
    SHELF_ID NUMBER(4) CONSTRAINT pk_shelf PRIMARY KEY,
    FLOOR_NO NUMBER(2) NOT NULL,
    AISLE_CODE VARCHAR2(5) NOT NULL,
    CATEGORY VARCHAR2(50) NOT NULL
);

```

☒ Autocommit Rows: 10 Save Run

SELECT * FROM SHELF

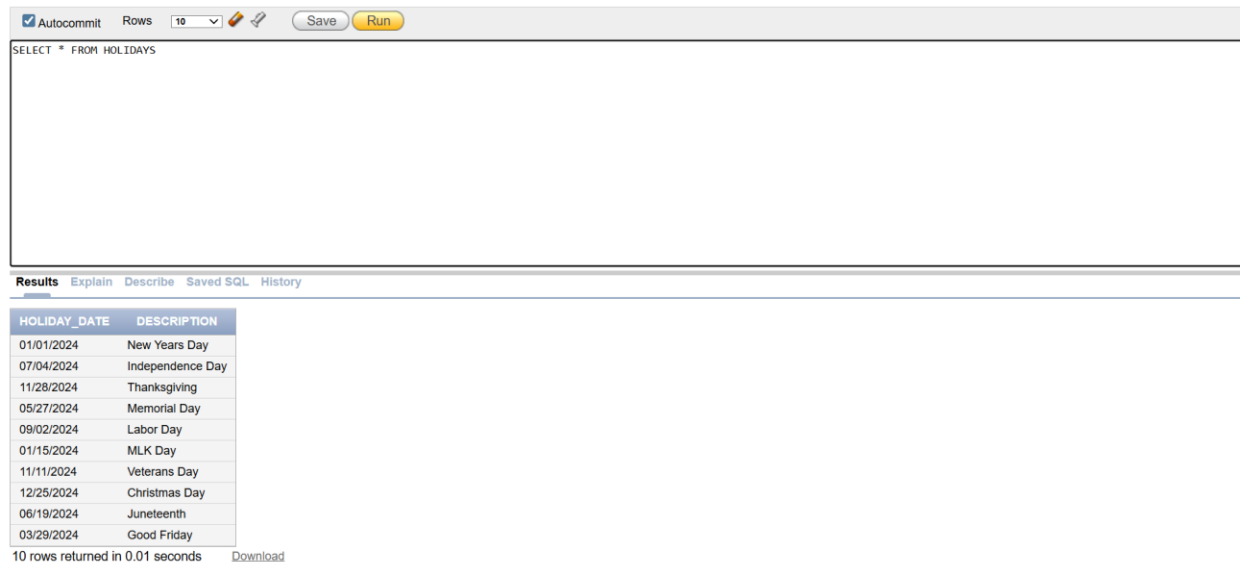
[Results](#) [Explain](#) [Describe](#) [Saved SQL](#) [History](#)

SHELF_ID	FLOOR_NO	AISLE_CODE	CATEGORY
101	1	A1	Computer Science
102	1	A2	Mathematics
302	3	C2	Science Fiction
103	1	A3	Engineering
203	2	B3	Philosophy
401	4	D1	Reference
402	4	D2	Periodicals
201	2	B1	Fiction
202	2	B2	History
301	3	C1	Biography

10 rows returned in 0.01 seconds [Download](#)

Application Express 4.0.2.00.09

```
CREATE TABLE HOLIDAYS (
    HOLIDAY_DATE DATE CONSTRAINT pk_holiday PRIMARY KEY,
    DESCRIPTION VARCHAR2(50)
);
```



The screenshot shows a database application window. At the top, there is a toolbar with a checked 'Autocommit' checkbox, a 'Rows' dropdown set to '10', and 'Save' and 'Run' buttons. Below the toolbar, a SQL query is entered in a text area: 'SELECT * FROM HOLIDAYS'. Below the query area, there is a tabbed interface with 'Results' selected. The results are displayed in a table with two columns: 'HOLIDAY_DATE' and 'DESCRIPTION'. The table contains 10 rows of data. Below the table, a status bar indicates '10 rows returned in 0.01 seconds' and a 'Download' button.

HOLIDAY_DATE	DESCRIPTION
01/01/2024	New Years Day
07/04/2024	Independence Day
11/28/2024	Thanksgiving
05/27/2024	Memorial Day
09/02/2024	Labor Day
01/15/2024	MLK Day
11/11/2024	Veterans Day
12/25/2024	Christmas Day
06/19/2024	Juneteenth
03/29/2024	Good Friday

10 rows returned in 0.01 seconds [Download](#)

Application Express 4.0.2.00.09

Step 2: Bibliographic Tables

```
CREATE TABLE PUBLISHER (
    PUB_ID NUMBER(5) CONSTRAINT pk_pub PRIMARY KEY,
    PUB_NAME VARCHAR2(100) NOT NULL CONSTRAINT uq_pub_name UNIQUE,
    ADDRESS VARCHAR2(255),
    CONTACT_NO VARCHAR2(20)
);
```

☒ Autocommit Rows: 10   Save Run

SELECT * FROM PUBLISHER

Results Explain Describe Saved SQL History

PUB_ID	PUB_NAME	ADDRESS	CONTACT_NO
1	O'Reilly Media	California, USA	555-0101
3	Penguin Random House	New York, USA	555-0103
4	HarperCollins	New York, USA	555-0104
5	Springer	Berlin, Germany	555-0105
6	Oxford University Press	Oxford, UK	555-0106
7	MIT Press	Massachusetts, USA	555-0107
8	Packt Publishing	Birmingham, UK	555-0108
2	Pearson	London, UK	555-0102
9	Macmillan	London, UK	555-0109
10	Wiley	New Jersey, USA	555-0110

10 rows returned in 0.02 seconds [Download](#)

Application Express 4.0.2.00.09

```
CREATE TABLE AUTHOR (  
  
    AUTHOR_ID NUMBER(10) CONSTRAINT pk_author PRIMARY KEY,  
  
    FULL_NAME VARCHAR2(100) NOT NULL,  
  
    BIO VARCHAR2(1000)  
  
);
```

☒ Autocommit Rows: 10   Save Run

SELECT * FROM AUTHOR

Results Explain Describe Saved SQL History

AUTHOR_ID	FULL_NAME	BIO
1	Robert C. Martin	Known as Uncle Bob, author of Clean Code.
2	J.K. Rowling	Author of the Harry Potter series.
3	George R.R. Martin	Author of A Song of Ice and Fire.
4	Thomas H. Cormen	Co-author of Introduction to Algorithms.
5	Stephen King	Prolific American author of horror and suspense.
6	Yuval Noah Harari	Historian and author of Sapiens.
7	Carl Sagan	Astronomer and science communicator.
8	Margaret Atwood	Canadian poet and novelist.
9	Neil Gaiman	Writer of short fiction and graphic novels.
10	Walter Isaacson	Author of various famous biographies.

10 rows returned in 0.02 seconds [Download](#)

Application Express 4.0.2.00.09

```

CREATE TABLE BOOK (
    ISBN VARCHAR2(13) CONSTRAINT pk_book PRIMARY KEY,
    TITLE VARCHAR2(255) NOT NULL,
    PUB_ID NUMBER(5),
    PUB_YEAR NUMBER(4),
    EDITION NUMBER(2) DEFAULT 1,
    CONSTRAINT fk_book_pub FOREIGN KEY (PUB_ID) REFERENCES PUBLISHER(PUB_ID),
    CONSTRAINT chk_isbn_format CHECK (REGEXP_LIKE(ISBN, '^(97(8|9))?\d{9}(\d|X)$'))
);

```

Autocommit Rows: 10 Save Run

SELECT * FROM BOOK

Results Explain Describe Saved SQL History

ISBN	TITLE	PUB_ID	PUB_YEAR	EDITION
9780553103540	A Game of Thrones	3	1996	1
9781491901632	Learning Python	1	2013	5
9780143111580	The Odyssey	3	2016	1
9781501163401	Steve Jobs	10	2011	1
9780132350884	Clean Code	2	2008	1
9780439023528	The Hunger Games	3	2008	1
9780262033848	Introduction to Algorithms	7	2009	3
9780062317094	Sapiens	4	2011	1
9780345339683	The Hobbit	4	1937	1
9781107057135	Quantum Mechanics	6	2014	2

10 rows returned in 0.01 seconds Download

Application Express 4.0.2.00.09

```

CREATE TABLE BOOK_AUTHOR (
    ISBN VARCHAR2(13),
    AUTHOR_ID NUMBER(10),
    CONSTRAINT pk_book_author PRIMARY KEY (ISBN, AUTHOR_ID),
    CONSTRAINT fk_ba_isbn FOREIGN KEY (ISBN) REFERENCES BOOK(ISBN) ON DELETE CASCADE,
    CONSTRAINT fk_ba_author FOREIGN KEY (AUTHOR_ID) REFERENCES AUTHOR(AUTHOR_ID)
);

```

);

☒ Autocommit Rows: 10 Save Run

SELECT * FROM BOOK_AUTHOR

Results Explain Describe Saved SQL History

ISBN	AUTHOR_ID
9780062317094	6
9780132350884	1
9780143111580	6
9780262033848	4
9780345339683	9
9780439023528	8
9780553103540	3
9781107057135	7
9781491901632	4
9781501163401	10

10 rows returned in 0.01 seconds [Download](#)

Application Express 4.0.2.00.09

Step 3: Inventory and Member Tables

```
CREATE TABLE MEMBER (  
    MEMBER_ID NUMBER(10) CONSTRAINT pk_member PRIMARY KEY,  
    NAME VARCHAR2(100) NOT NULL,  
    EMAIL VARCHAR2(100) NOT NULL CONSTRAINT uq_mem_email UNIQUE,  
    PHONE VARCHAR2(15) NOT NULL,  
    TYPE_ID NUMBER(2) NOT NULL,  
    STATUS VARCHAR2(10) DEFAULT 'ACTIVE',  
    REG_DATE DATE DEFAULT SYSDATE,  
    CONSTRAINT fk_mem_type FOREIGN KEY (TYPE_ID) REFERENCES  
MEMBERSHIP_TYPE(TYPE_ID),  
    CONSTRAINT chk_mem_status CHECK (STATUS IN ('ACTIVE', 'SUSPENDED', 'EXPIRED'))  
);
```


☒ Autocommit	Rows	10			Save	Run
--	------	----	--	--	------	-----

SELECT * FROM MEMBER

Results Explain Describe Saved SQL History

MEMBER_ID	NAME	EMAIL	PHONE	TYPE_ID	STATUS	REG_DATE
1001	Alice Smith	alice@email.com	1234567890	1	ACTIVE	12/20/2025
1002	Bob Jones	bob@email.com	2345678901	2	ACTIVE	12/20/2025
1003	Charlie Brown	charlie@email.com	3456789012	1	ACTIVE	12/20/2025
1004	Diana Prince	diana@email.com	4567890123	3	ACTIVE	12/20/2025
1005	Edward Norton	edward@email.com	5678901234	5	ACTIVE	12/20/2025
1006	Fiona Glenanne	fiona@email.com	6789012345	1	SUSPENDED	12/20/2025
1007	George Costanza	george@email.com	7890123456	4	ACTIVE	12/20/2025
1008	Hannah Abbott	hannah@email.com	8901234567	6	ACTIVE	12/20/2025
1009	Ian Wright	ian@email.com	9012345678	2	EXPIRED	12/20/2025
1010	Jane Doe	jane@email.com	0123456789	1	ACTIVE	12/20/2025

10 rows returned in 0.01 seconds [Download](#)

Application Express 4.0.2.00.09

```

CREATE TABLE BOOK_COPY (
  COPY_ID NUMBER(12) CONSTRAINT pk_copy PRIMARY KEY,
  ISBN VARCHAR2(13) NOT NULL,
  SHELF_ID NUMBER(4) NOT NULL,
  STATUS VARCHAR2(20) DEFAULT 'AVAILABLE',
  CONSTRAINT fk_copy_isbn FOREIGN KEY (ISBN) REFERENCES BOOK(ISBN),
  CONSTRAINT fk_copy_shelf FOREIGN KEY (SHELF_ID) REFERENCES SHELF(SHELF_ID),
  CONSTRAINT chk_copy_status CHECK (STATUS IN ('AVAILABLE', 'ISSUED', 'LOST',
'MAINTENANCE', 'RESERVED'))
);

```

Autocommit Rows 10 Save Run

SELECT * FROM BOOK_COPY

Results Explain Describe Saved SQL History

COPY_ID	ISBN	SHELF_ID	STATUS
5001	9780132350884	101	ISSUED
5002	9780132350884	101	ISSUED
5003	9780262033848	101	AVAILABLE
5004	9780553103540	201	AVAILABLE
5005	9780062317094	202	RESERVED
5006	9781501163401	301	AVAILABLE
5007	9780439023528	201	LOST
5008	9781491901632	101	MAINTENANCE
5009	9781107057135	102	AVAILABLE
5010	9780345339683	201	AVAILABLE

10 rows returned in 0.01 seconds Download

Application Express 4.0.2.00.09

Step 4: Transactional Tables

```
CREATE TABLE LOAN_TRANSACTION (
  TRANS_ID NUMBER(15) CONSTRAINT pk_loan PRIMARY KEY,
  COPY_ID NUMBER(12) NOT NULL,
  MEMBER_ID NUMBER(10) NOT NULL,
  ISSUE_DATE DATE DEFAULT SYSDATE NOT NULL,
  DUE_DATE DATE NOT NULL,
  RETURN_DATE DATE,
  STATUS VARCHAR2(10) DEFAULT 'OPEN',
  CONSTRAINT fk_loan_copy FOREIGN KEY (COPY_ID) REFERENCES BOOK_COPY(COPY_ID),
  CONSTRAINT fk_loan_member FOREIGN KEY (MEMBER_ID) REFERENCES
MEMBER(MEMBER_ID),
  CONSTRAINT chk_loan_status CHECK (STATUS IN ('OPEN', 'CLOSED'))
);
```

Autocommit

Rows

10



Save

Run

SELECT * FROM LOAN_TRANSACTION

Results

Explain

Describe

Saved SQL

History

TRANS_ID	COPY_ID	MEMBER_ID	ISSUE_DATE	DUE_DATE	RETURN_DATE	STATUS
1	5002	1001	12/15/2025	12/29/2025	-	OPEN
2	5005	1002	11/30/2025	12/14/2025	-	OPEN
3	5001	1003	12/10/2025	12/24/2025	12/18/2025	CLOSED
4	5003	1004	12/19/2025	01/02/2026	-	OPEN
5	5006	1005	12/05/2025	03/05/2026	-	OPEN
6	5010	1001	11/20/2025	12/04/2025	12/05/2025	CLOSED
7	5004	1007	12/18/2025	01/01/2026	-	OPEN
8	5009	1010	12/15/2025	12/29/2025	-	OPEN
9	5002	1003	11/10/2025	11/24/2025	11/30/2025	CLOSED
10	5008	1002	10/21/2025	01/19/2026	12/10/2025	CLOSED

More than 10 rows available. Increase rows selector to view more rows.

10 rows returned in 0.02 seconds

Download

```
CREATE TABLE FINE (
    FINE_ID NUMBER(15) CONSTRAINT pk_fine PRIMARY KEY,
    TRANS_ID NUMBER(15) NOT NULL,
    AMOUNT NUMBER(8,2) NOT NULL,
    PAID_STATUS VARCHAR2(10) DEFAULT 'UNPAID',
    PAY_DATE DATE,
    CONSTRAINT fk_fine_loan FOREIGN KEY (TRANS_ID) REFERENCES
    LOAN_TRANSACTION(TRANS_ID),
    CONSTRAINT chk_fine_status CHECK (PAID_STATUS IN ('PAID', 'UNPAID'))
);
```

☒ Autocommit Rows: 10 Save Run

SELECT * FROM FINE

Results Explain Describe Saved SQL History

FINE_ID	TRANS_ID	AMOUNT	PAID_STATUS	PAY_DATE
8	5	0	UNPAID	-
9	7	0	UNPAID	-
10	8	0	UNPAID	-
1	2	7	UNPAID	-
2	6	4.5	PAID	12/06/2025
3	9	10	PAID	12/01/2025
4	10	0	PAID	12/10/2025
5	1	0	UNPAID	-
6	3	0	UNPAID	-
7	4	0	UNPAID	-

10 rows returned in 0.02 seconds [Download](#)

Application Express 4.0.2.00.09

```
CREATE TABLE RESERVATION (  
    RES_ID NUMBER(10) PRIMARY KEY,  
    ISBN VARCHAR2(13) NOT NULL REFERENCES BOOK(ISBN),  
    MEMBER_ID NUMBER(10) NOT NULL REFERENCES MEMBER(MEMBER_ID),  
    RES_DATE DATE DEFAULT SYSDATE,  
    STATUS VARCHAR2(20) DEFAULT 'PENDING' CHECK (STATUS IN ('PENDING', 'FULFILLED',  
'CANCELLED'))  
);
```

☒ Autocommit	Rows	10			Save	Run
--	------	----	--	--	------	-----

SELECT * FROM RESERVATION

Results
Explain
Describe
Saved SQL
History

RES_ID	ISBN	MEMBER_ID	RES_DATE	STATUS
1	9780062317094	1005	12/19/2025	PENDING
2	9780132350884	1001	12/18/2025	FULFILLED
3	9780262033848	1002	12/17/2025	CANCELLED
4	9780553103540	1003	12/20/2025	PENDING
5	9781501163401	1004	12/15/2025	PENDING
6	9780439023528	1007	12/19/2025	PENDING
7	9781491901632	1010	12/16/2025	FULFILLED
8	9781107057135	1008	12/10/2025	CANCELLED
9	9780345339683	1002	12/18/2025	PENDING
10	9780143111580	1001	12/12/2025	FULFILLED

10 rows returned in 0.01 seconds
Download

Application Express 4.0.2.00.09

7. Procedural Logic (PL/SQL Implementation)

The defining characteristic of an enterprise LMS is the embedding of business logic within the database layer. This ensures that rules are enforced consistently, protecting data integrity regardless of the application interface used.

7.1 Advanced Fine Calculation Logic

One of the most complex requirements is the calculation of fines while excluding non-working days. This requires a PL/SQL function that iterates through the overdue period, checking against weekends and the HOLIDAYS table.

Function: FN_CALC_OVERDUE_DAYS This function accepts a due date and a return date, returning the number of "fineable" days.

CREATE OR REPLACE FUNCTION FN_CALC_OVERDUE_DAYS(p_due_date DATE, p_return_date DATE)

RETURN NUMBER IS

v_days NUMBER := 0;

v_curr_date DATE;

v_is_holiday NUMBER;

BEGIN

-- If returned on time or early, no fine days

IF p_return_date <= p_due_date THEN

 RETURN 0;

END IF;

-- Start counting from the day after the due date

v_curr_date := p_due_date + 1;

WHILE v_curr_date <= p_return_date LOOP

 -- Check if the day is a weekend (Safe check using DY format)

 -- Note: NLS settings can affect this, so explicit English check is safer for this example

 IF TO_CHAR(v_curr_date, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH') NOT IN ('SAT', 'SUN') THEN

 -- Check if the day is in the HOLIDAYS table

 SELECT COUNT(*) INTO v_is_holiday

 FROM HOLIDAYS

 WHERE HOLIDAY_DATE = TRUNC(v_curr_date);

 -- If not a holiday, increment the fineable day count

 IF v_is_holiday = 0 THEN

 v_days := v_days + 1;

 END IF;

 END IF;

-- Move to the next day

```

        v_curr_date := v_curr_date + 1;

END LOOP;

RETURN v_days;

EXCEPTION

    WHEN OTHERS THEN

        RETURN 0; -- Fail safe

END;
```

7.2 Enforcing Borrowing Limits via Triggers

To prevent a member from exceeding their quota (e.g., a student borrowing a 6th book), a BEFORE INSERT trigger checks the count of currently 'OPEN' loans for that member against their membership type limit.

```

CREATE OR REPLACE TRIGGER TRG_ENFORCE_LOAN_LIMIT
BEFORE INSERT ON LOAN_TRANSACTION
FOR EACH ROW
DECLARE
    v_current_loans NUMBER;
    v_max_allowed NUMBER;
BEGIN
    -- 1. Retrieve the maximum allowed books for this member's type
    SELECT mt.MAX_BOOKS
    INTO v_max_allowed
    FROM MEMBER m
    JOIN MEMBERSHIP_TYPE mt ON m.TYPE_ID = mt.TYPE_ID
    WHERE m.MEMBER_ID = :NEW.MEMBER_ID;
```

```

-- 2. Count the currently open loans for this member

SELECT COUNT(*)
INTO v_current_loans
FROM LOAN_TRANSACTION
WHERE MEMBER_ID = :NEW.MEMBER_ID AND STATUS = 'OPEN';

-- 3. Validation Logic

IF v_current_loans >= v_max_allowed THEN
    RAISE_APPLICATION_ERROR(-20001,
        'Borrowing limit reached. Member cannot borrow more than ' || v_max_allowed || ' items.');
```

END IF;

END;

7.3 Automated Status Synchronization

When a book is issued, its status in the inventory (BOOK_COPY) must immediately change to 'ISSUED' to prevent it from being loaned to another user. Conversely, upon return, it must revert to 'AVAILABLE'.

```

CREATE OR REPLACE TRIGGER TRG_SYNC_BOOK_STATUS
AFTER INSERT OR UPDATE ON LOAN_TRANSACTION
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        -- Mark copy as ISSUED upon new loan creation
        UPDATE BOOK_COPY
        SET STATUS = 'ISSUED'
```



```

WHERE COPY_ID = :NEW.COPY_ID;

ELSIF UPDATING AND :NEW.STATUS = 'CLOSED' THEN
    -- Mark copy as AVAILABLE upon loan closure
    UPDATE BOOK_COPY
    SET STATUS = 'AVAILABLE'
    WHERE COPY_ID = :NEW.COPY_ID;
END IF;
END;

```

7.4 Transactional Procedures

Direct SQL inserts into the LOAN_TRANSACTION table are risky because they might bypass necessary checks (like calculating the correct due date). Stored procedures provide a controlled interface for these operations.

Procedure: ISSUE_BOOK This procedure encapsulates the logic for creating a new loan. It calculates the Due_Date based on the member's LOAN_DAYS parameter.

```

CREATE OR REPLACE PROCEDURE PROC_ISSUE_BOOK(
    p_member_id IN NUMBER,
    p_copy_id IN NUMBER
) IS
    v_copy_status VARCHAR2(20);
    v_loan_days NUMBER;
    v_due_date DATE;
    v_mem_status VARCHAR2(10);
BEGIN
    -- 1. Validate Member Status
    SELECT STATUS INTO v_mem_status FROM MEMBER WHERE MEMBER_ID = p_member_id;

```

```

IF v_mem_status!= 'ACTIVE' THEN

    RAISE_APPLICATION_ERROR(-20002, 'Member account is Suspended or Expired.');
```

END IF;

-- 2. Validate Copy Availability

```

SELECT STATUS INTO v_copy_status FROM BOOK_COPY WHERE COPY_ID = p_copy_id;

IF v_copy_status!= 'AVAILABLE' THEN

    RAISE_APPLICATION_ERROR(-20003, 'Book Copy is not available for circulation.');
```

END IF;

-- 3. Calculate Due Date based on Membership Type

```

SELECT mt.LOAN_DAYS

INTO v_loan_days

FROM MEMBER m

JOIN MEMBERSHIP_TYPE mt ON m.TYPE_ID = mt.TYPE_ID

WHERE m.MEMBER_ID = p_member_id;
```

v_due_date := TRUNC(SYSDATE) + v_loan_days;

-- 4. Create Transaction (Triggers will handle Limit Check and Status Update)

```

INSERT INTO LOAN_TRANSACTION (TRANS_ID, COPY_ID, MEMBER_ID, DUE_DATE)

VALUES (seq_trans_id.NEXTVAL, p_copy_id, p_member_id, v_due_date);
```

COMMIT;

DBMS_OUTPUT.PUT_LINE('Book issued successfully. Due Date: ' |

```
| v_due_date);
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        ROLLBACK;
```

```
        RAISE_APPLICATION_ERROR(-20004, 'Invalid Member ID or Book Copy ID.');
```

```
    WHEN OTHERS THEN
```

```
        ROLLBACK;
```

```
        RAISE; -- Propagate error to application
```

```
END;
```

Procedure: RETURN_BOOK This procedure handles the check-in process, calculates fines if the book is overdue, and closes the transaction.

```
CREATE OR REPLACE PROCEDURE PROC_RETURN_BOOK(
```

```
    p_trans_id IN NUMBER
```

```
) IS
```

```
    v_due_date DATE;
```

```
    v_member_id NUMBER;
```

```
    v_overdue_days NUMBER;
```

```
    v_daily_fine NUMBER;
```

```
    v_fine_amt NUMBER;
```

```
    v_loan_status VARCHAR2(10);
```

```
BEGIN
```

```
    -- 1. Retrieve Loan Details
```

```
    SELECT DUE_DATE, MEMBER_ID, STATUS
```

```
    INTO v_due_date, v_member_id, v_loan_status
```

```
    FROM LOAN_TRANSACTION
```

```
WHERE TRANS_ID = p_trans_id;
```

```
IF v_loan_status = 'CLOSED' THEN
```

```
    RAISE_APPLICATION_ERROR(-20005, 'This transaction is already closed.');
```

```
END IF;
```

```
-- 2. Calculate Fines (using the Helper Function)
```

```
v_overdue_days := FN_CALC_OVERDUE_DAYS(v_due_date, TRUNC(SYSDATE));
```

```
IF v_overdue_days > 0 THEN
```

```
    -- Fetch Fine Rate for this member
```

```
    SELECT mt.DAILY_FINE
```

```
    INTO v_daily_fine
```

```
    FROM MEMBER m
```

```
    JOIN MEMBERSHIP_TYPE mt ON m.TYPE_ID = mt.TYPE_ID
```

```
    WHERE m.MEMBER_ID = v_member_id;
```

```
v_fine_amt := v_overdue_days * v_daily_fine;
```

```
-- Insert Fine Record
```

```
INSERT INTO FINE (FINE_ID, TRANS_ID, AMOUNT)
```

```
VALUES (seq_fine_id.NEXTVAL, p_trans_id, v_fine_amt);
```

```
DBMS_OUTPUT.PUT_LINE('Book returned overdue. Fine Generated: ' |
```

```
| v_fine_amt);
```

```
ELSE
    DBMS_OUTPUT.PUT_LINE('Book returned on time. No fine.');
```

END IF;

-- 3. Close the Loan Transaction

```
UPDATE LOAN_TRANSACTION
SET RETURN_DATE = SYSDATE,
    STATUS = 'CLOSED'
WHERE TRANS_ID = p_trans_id;
```

COMMIT;

EXCEPTION

```
WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20006, 'Transaction ID not found.');
```

WHEN OTHERS THEN

```
    ROLLBACK;
    RAISE;
```

END;

8. Reporting and Analytical Views

Views abstract the complexity of the underlying normalized schema, presenting "flat" data structures that are easier for application developers and reporting tools to consume.

8.1 Public Catalog View

This view aggregates data from BOOK, AUTHOR, and PUBLISHER to provide a searchable master list. It uses LISTAGG to concatenate multiple author names into a single string for display.

```
CREATE OR REPLACE VIEW VIEW_CATALOG_MASTER AS
SELECT
    b.ISBN,
    b.TITLE,
    b.EDITION,
    p.PUB_NAME,
    b.PUB_YEAR,
    LISTAGG(a.FULL_NAME, ', ') WITHIN GROUP (ORDER BY a.FULL_NAME) AS AUTHORS,
    (SELECT COUNT(*) FROM BOOK_COPY bc WHERE bc.ISBN = b.ISBN AND bc.STATUS =
'AVAILABLE') AS COPIES_AVAILABLE,
    (SELECT COUNT(*) FROM BOOK_COPY bc WHERE bc.ISBN = b.ISBN) AS TOTAL_COPIES
FROM BOOK b
JOIN PUBLISHER p ON b.PUB_ID = p.PUB_ID
JOIN BOOK_AUTHOR ba ON b.ISBN = ba.ISBN
JOIN AUTHOR a ON ba.AUTHOR_ID = a.AUTHOR_ID
GROUP BY b.ISBN, b.TITLE, b.EDITION, p.PUB_NAME, b.PUB_YEAR;
```

☒ Autocommit

Rows 10

Save

Run

SELECT * FROM VIEW_CATALOG_MASTER

Results

Explain

Describe

Saved SQL

History

ISBN	TITLE	EDITION	PUB_NAME	PUB_YEAR	AUTHORS	COPIES_AVAILABLE	TOTAL_COPIES
9780062317094	Sapiens	1	HarperCollins	2011	Yuval Noah Harari	0	1
9780132350884	Clean Code	1	Pearson	2008	Robert C. Martin	0	2
9780143111580	The Odyssey	1	Penguin Random House	2016	Yuval Noah Harari	0	0
9780262033848	Introduction to Algorithms	3	MIT Press	2009	Thomas H. Cormen	1	1
9780345339683	The Hobbit	1	HarperCollins	1937	Neil Gaiman	1	1
9780439023528	The Hunger Games	1	Penguin Random House	2008	Margaret Atwood	0	1
9780553103540	A Game of Thrones	1	Penguin Random House	1996	George R.R. Martin	1	1
9781107057135	Quantum Mechanics	2	Oxford University Press	2014	Carl Sagan	1	1
9781491901632	Learning Python	5	O Reilly Media	2013	Thomas H. Cormen	0	1
9781501163401	Steve Jobs	1	Wiley	2011	Walter Isaacson	1	1

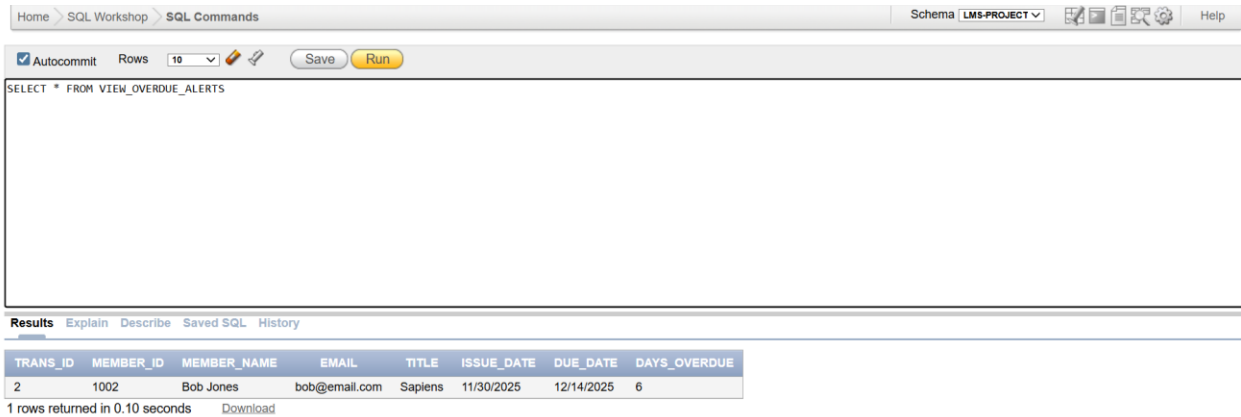
10 rows returned in 0.08 seconds

Download

8.2 Overdue Alerts View

This view allows the administration to identify members who are holding books past their due date. It calculates the days overdue dynamically.

```
CREATE OR REPLACE VIEW VIEW_OVERDUE_ALERTS AS
SELECT
    l.TRANS_ID,
    m.MEMBER_ID,
    m.NAME AS MEMBER_NAME,
    m.EMAIL,
    b.TITLE,
    l.ISSUE_DATE,
    l.DUE_DATE,
    TRUNC(SYSDATE - l.DUE_DATE) AS DAYS_OVERDUE
FROM LOAN_TRANSACTION l
JOIN MEMBER m ON l.MEMBER_ID = m.MEMBER_ID
JOIN BOOK_COPY bc ON l.COPY_ID = bc.COPY_ID
JOIN BOOK b ON bc.ISBN = b.ISBN
WHERE l.STATUS = 'OPEN'
AND l.DUE_DATE < SYSDATE;
```



8.3 Financial Defaulters Report

CREATE OR REPLACE VIEW VIEW_FINANCIAL_DEFAULTERS AS

m.MEMBER_ID,

m.EMAIL,

FROM MEMBER m

JOIN FINE f ON l.TRANS_ID = f.TRANS_ID

GROUP BY m.MEMBER_ID, m.NAME, m.EMAIL

ORDER BY TOTAL_UNPAID_FINE DESC;

Home > SQL Workshop > SQL Commands Schema LMS-PROJECT

☒ Autocommit Rows 10 Save Run

SELECT * FROM VIEW_FINANCIAL_DEFAULTERS

Results Explain Describe Saved SQL History

MEMBER_ID	NAME	EMAIL	TOTAL_UNPAID_FINE
1002	Bob Jones	bob@email.com	7

1 rows returned in 0.05 seconds Download

Application Express 4.0.2.00.09

9. Complex Reporting Queries

Beyond static views, the database supports ad-hoc queries for strategic decision-making.

Query 1: Most Popular Genres (Circulation Analysis) Analyzes which categories of books are circulating most frequently to guide future acquisition budgets.

```
SELECT
    s.CATEGORY,
    COUNT(l.TRANSACTION_ID) AS CIRCULATION_COUNT,
    ROUND(RATIO_TO_REPORT(COUNT(l.TRANSACTION_ID)) OVER () * 100, 2) AS PERCENTAGE_SHARE
FROM LOAN_TRANSACTION l
JOIN BOOK_COPY bc ON l.COPY_ID = bc.COPY_ID
JOIN SHELF s ON bc.SHELF_ID = s.SHELF_ID
GROUP BY s.CATEGORY
ORDER BY CIRCULATION_COUNT DESC;
```

Home > SQL Workshop > SQL Commands Schema LMS-PROJECT

☒ Autocommit Rows 10 Save Run

```

SELECT
  s.CATEGORY,
  COUNT(l.TRANS_ID) AS CIRCULATION_COUNT,
  ROUND(RATIO_TO_REPORT(COUNT(l.TRANS_ID)) OVER () * 100, 2) AS PERCENTAGE_SHARE
FROM LOAN_TRANSACTION l
JOIN BOOK_COPY bc ON l.COPY_ID = bc.COPY_ID
JOIN SHELF s ON bc.SHELF_ID = s.SHELF_ID
GROUP BY s.CATEGORY
ORDER BY CIRCULATION_COUNT DESC;

```

Results Explain Describe Saved SQL History

CATEGORY	CIRCULATION_COUNT	PERCENTAGE_SHARE
Computer Science	6	54.55
Fiction	2	18.18
History	1	9.09
Biography	1	9.09
Mathematics	1	9.09

5 rows returned in 0.03 seconds [Download](#)

Application Express 4.0.2.00.09

Query 2: Book Turnover Rate Identifies "Dead Stock" (books that have never been issued) to assist in weeding/archiving processes.

```

SELECT

  b.TITLE,

  b.ISBN,

  bc.COPY_ID,

  bc.STATUS,

  bc.SHELF_ID

FROM BOOK b

JOIN BOOK_COPY bc ON b.ISBN = bc.ISBN

WHERE NOT EXISTS (

  SELECT 1 FROM LOAN_TRANSACTION l WHERE l.COPY_ID = bc.COPY_ID

)

ORDER BY b.TITLE;

```

```
SELECT
    b.TITLE,
    b.ISBN,
    bc.COPY_ID,
    bc.STATUS,
    bc.SHELF_ID
FROM BOOK b
JOIN BOOK_COPY bc ON b.ISBN = bc.ISBN
WHERE NOT EXISTS (
    SELECT 1 FROM LOAN_TRANSACTION l WHERE l.COPY_ID = bc.COPY_ID
)
ORDER BY b.TITLE;
```

Results Explain Describe Saved SQL History

TITLE	ISBN	COPY_ID	STATUS	SHELF_ID
The Hunger Games	9780439023528	5007	LOST	201

1 rows returned in 0.02 seconds [Download](#)